Finiteness 有限性

Linear/ Sequential search 線性/循序搜尋

二分搜 => O(log n +1)

- 閉區間 [l, r]
  - while(l <= r)
  - l = mid + 1
  - r = mid - 1

Induction Hypothesis 歸納假設

O(n^3) < O(2^n)

O(n)	<=	
Ω(n)	>=	
θ(n)	==	
o(n)	<	n ∉ o(5n), 因 n 不小於 0.1 * 5n (c * n)

o(n) = O(n) – Ω(n)

expect g(n) = {n, n 為奇數; 1, n為偶數}

n = 99, g(99) = 1 => ∈O(n), ∉ Ω(n)

- MST 最小生成數：

所有點沒有環的路權重和最小

[直到邊數為 n - 1]

- Prim 內外(候選邊)：

任選開始，下一節點一端在當前樹內一端樹外挑權重最小 $\Theta(n^2)$

- Kruskal 分組：

排序(小)權重邊，先各自群組再加鄰點進群組, 已為同組邊捨棄$\Theta(m \lg m)$

Prim 和 Kruskal 的差別（常考問答）

	Prim	Kruskal
看的範圍	只看跟目前這棵樹相連的邊	看全部的邊
過程中	永遠是一棵連在一起的樹	可能是好幾塊（森林），最後才連成一棵
複雜度	$\Theta(n^2)$	$\Theta(m \lg m)$
適合	邊很多（稠密圖）	邊很少（稀疏圖）

邊多時 Prim 比較快

- Dijkstra 最短路徑：

更新當前節點相鄰路徑之最短值 Dijkstra 不能處理負的權重（貪婪「確定了就不回頭」會出錯）。



輪	確定	B	C	D	E
0	A	4	1	∞	∞
1	C	4→3	✓1	∞→6	∞→9
2	B	✓3	✓	6→4	9
3	D	✓	✓	✓4	9→7
4	E	✓	✓	✓	✓7

排程

- 最小化總系統時間: 耗時最短

- 截止期限排程: 同單位時間報酬最高 & 當前累積時段小於截止期限



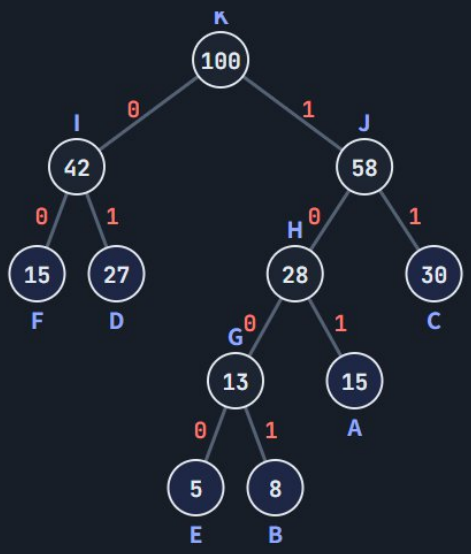
霍夫曼樹 編碼

- 不斷兩個最小節點合併，根結點走道節點為編碼

[權重 = 出現次數]

[總位元 = 權重 x 深度] (深度 == 長度)

霍夫曼樹 一步一步長　投影片 p.32　練習



步驟 6：只剩一棵樹，**結束**。左邊標 0、右邊標 1，從樹根走到樹葉就是字碼。
總位元 Σ(權重×字碼長度) = **241**；驗算：新節點權重和 13+28+42+58+100 = 241 ✓

← 上一步　下一步 →　重來　直接看答案　6 / 6

字元	權重	字碼	長度	權重×長度
F	15	00	2	30
D	27	01	2	54
E	5	1000	4	20
B	8	1001	4	32
A	15	101	3	45
C	30	11	2	60
總和				**241**

貪婪 vs 動態規劃

	貪婪	動態規劃 DP
做法	每步選當下最好，不回頭	把小問題答案存成表，組合出大問題
例子	Dijkstra $\Theta(n^2)$（單一起點）	Floyd $\Theta(n^3)$（所有點對）
優點	簡單、快	只要符合「最佳化原則」就一定對
缺點	很難確定會不會拿到最佳解，要另外證明	通常比較慢、要記很多表格

演算法	怎麼算出來（考試要會寫這行）	結果
Prim	repeat 跑 n−1 次，裡面 2 個 for 各跑 n−1 次 → $2(n-1)(n-1)$	$\Theta(n^2)$
Dijkstra	跟 Prim 完全一樣的迴圈結構 → $2(n-1)(n-1)$	$\Theta(n^2)$
Kruskal	排序 m 條邊要 m lg m；最差情況 m 條邊都要看過	$\Theta(m \lg m)$
截止期限排程	第 i 個工作：插入要 i−1 次比較、檢查可行要 i 次 → $\sum_{i=2}^{n}(2i-1) = n^2 - 1$	$\Theta(n^2)$
0-1 背包 DP	整張表 n 列 × W 行；只算需要的格子最多 $2^n - 1$	$O(\min(2^n, nW))$
Floyd（對照）	三層迴圈	$\Theta(n^3)$

回朔

第 4 題（0-1 背包 DP）

W = 7；物品①1kg$1、②3kg$4、③4kg$5、④5kg$7。(a) 畫出 P 表，求最大價值與拿哪些。(b) 如果用「每公斤最值錢先拿」的貪婪，會得到多少？(c) 如果是 Fractional 呢？

▾ **第 4 題答案**

i \ w	0	1	2	3	4	5	6	7
0	0	0	0	0	0	0	0	0
1	0	1	1	1	1	1	1	1
2	0	1	1	4	5	5	5	5
3	0	1	1	4	5	6	6	9
4	0	1	1	4	5	7	8	9

(a) P[4][7] = max(P[3][7]=9, 7+P[3][2]=8) = **9**。回溯：P[4][7]=P[3][7] → ④沒拿；P[3][7]=9≠P[2][7]=5 → 拿③，w=3；P[2][3]=4≠P[1][3]=1 → 拿②，w=0。答案 {②,③}，9 元。

(b) 每公斤：①1、②1.33、③1.25、④1.4 → 先拿④(5kg)，剩 2kg → ②③放不下 → 拿①。共 **8 元**，比 9 少，貪婪失敗。

(c) Fractional：④全部(5kg,$7)，剩 2kg 拿②的 2/3 → 4×2/3 ≈ 2.67。共 **≈ 9.67 元**。

完整順序（小到大）：$1 < \lg n < n < n \lg n < n^2 < n^3 < 2^n < n!$

口訣：**常數 → 對數 → 線性 → n lg n → 多項式 → 指數 → 階乘。**

二分搜最壞 => 不斷往右找

(2) 稀疏 vs 稠密

	Prim	Kruskal Θ(m lg m)	誰好
(a) 很稀疏：$m \approx n - 1$	$\Theta(n^2)$	$\Theta(n \lg n)$	**Kruskal**
(b) 很稠密：$m \approx n(n-1)/2$	$\Theta(n^2)$	$\Theta(n^2 \lg n)$	**Prim**

▾ **考題：DP 和分治法的相同與不同？（必背）**

相同：都把問題切成較小、性質相同的子問題。

不同：分治法 top-down，子問題各自獨立時有效率，但子問題重複時會一直重算；DP bottom-up，先解最小的子問題並**存起來**，之後直接查表，所以子問題重複時也有效率。